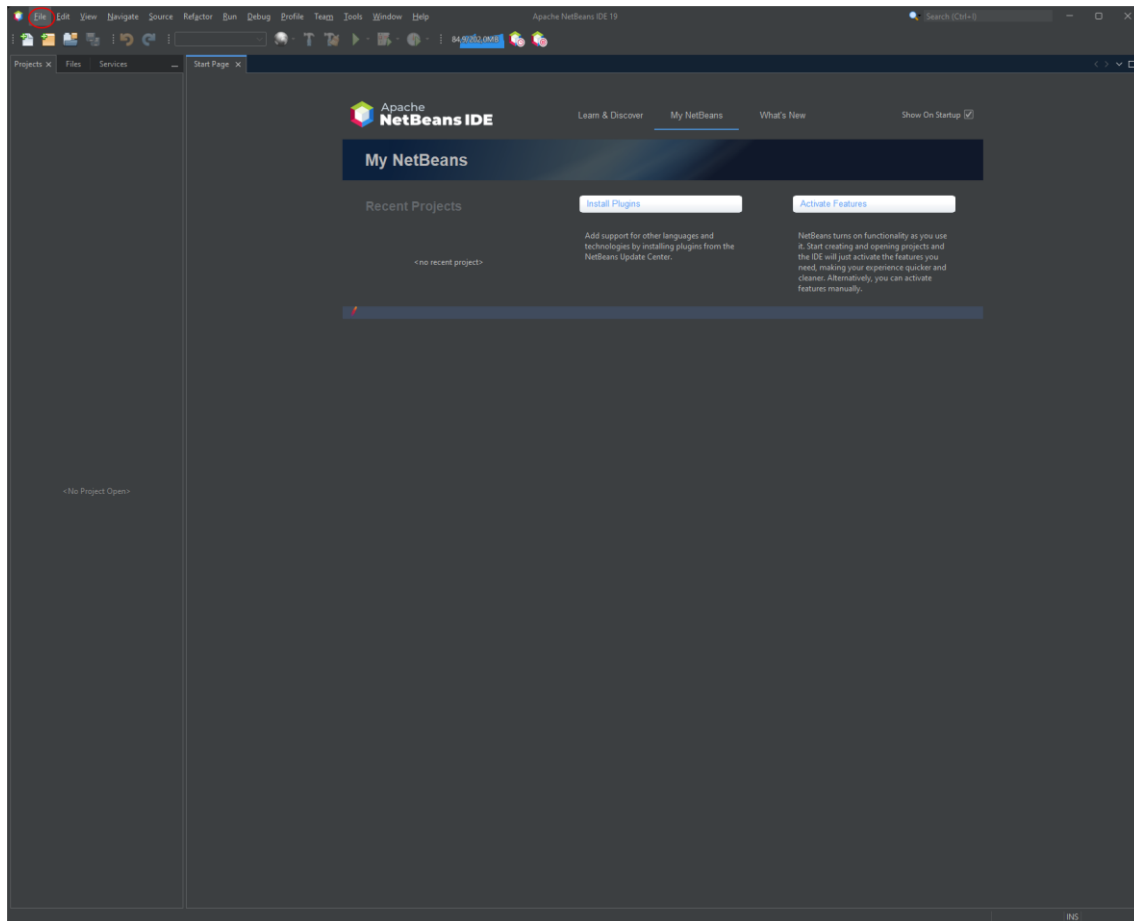


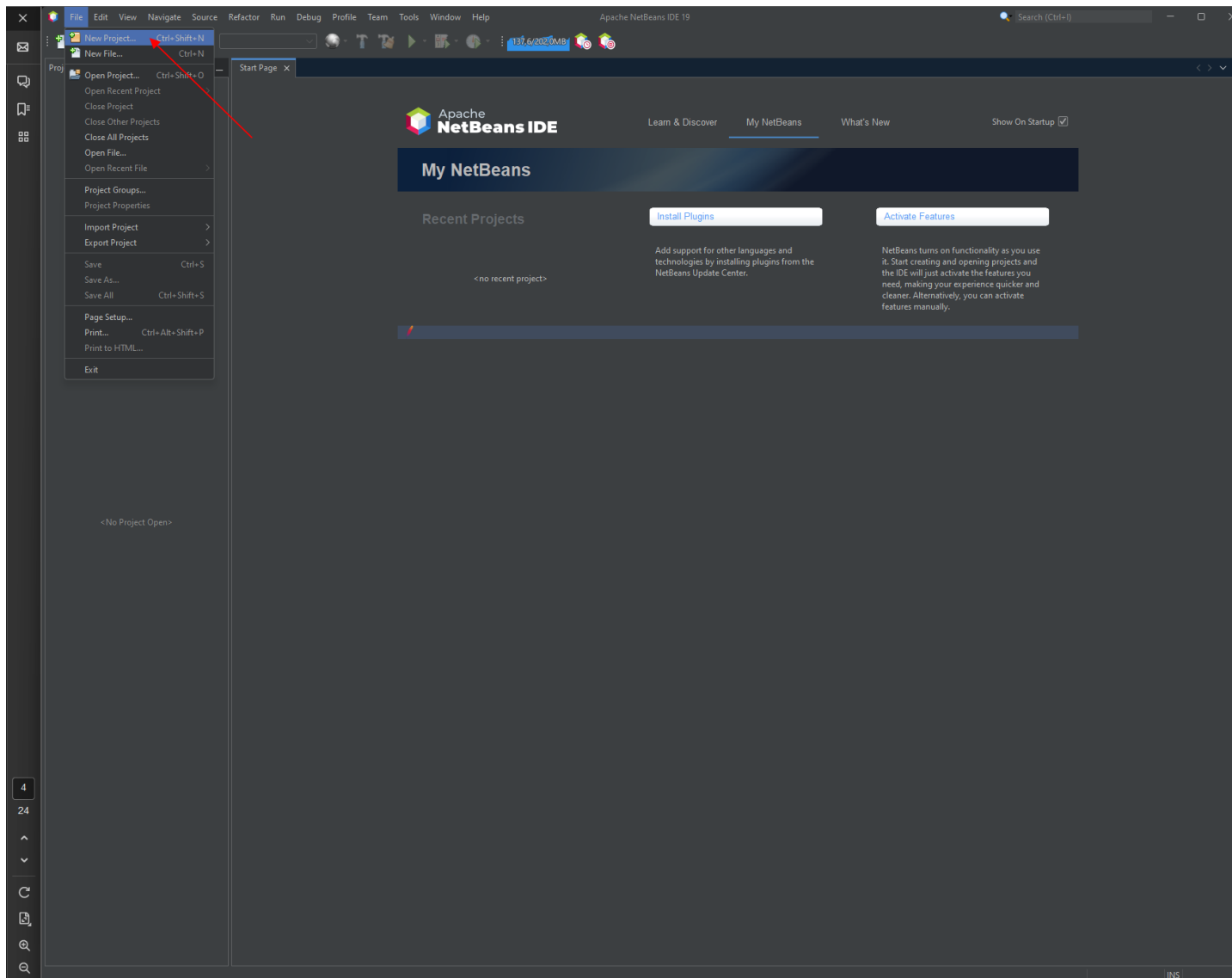
UT-01: ESTRUCTURAS Y ELEMENTOS DE UN PROGRAMA

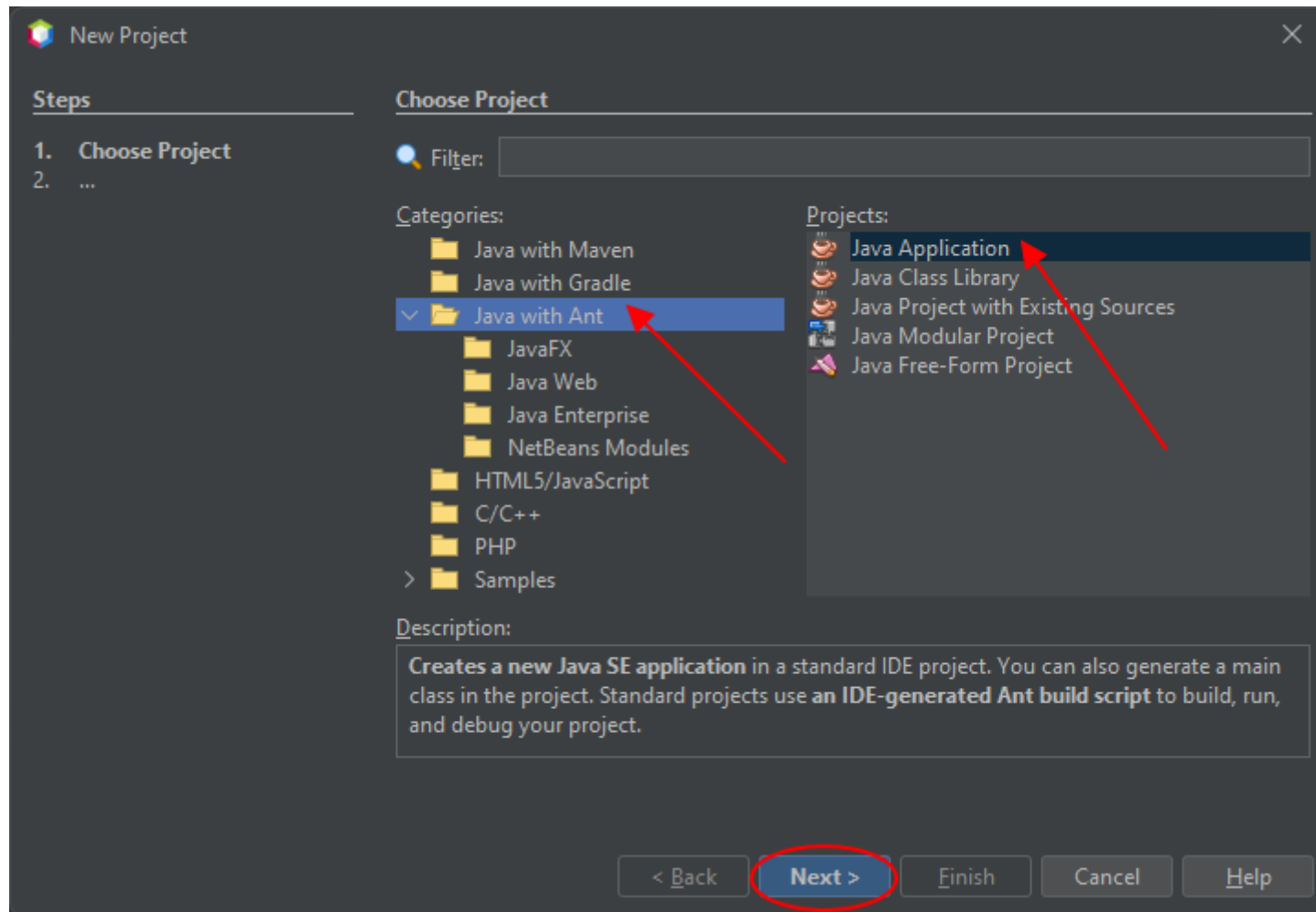
¿Qué vamos a ver?

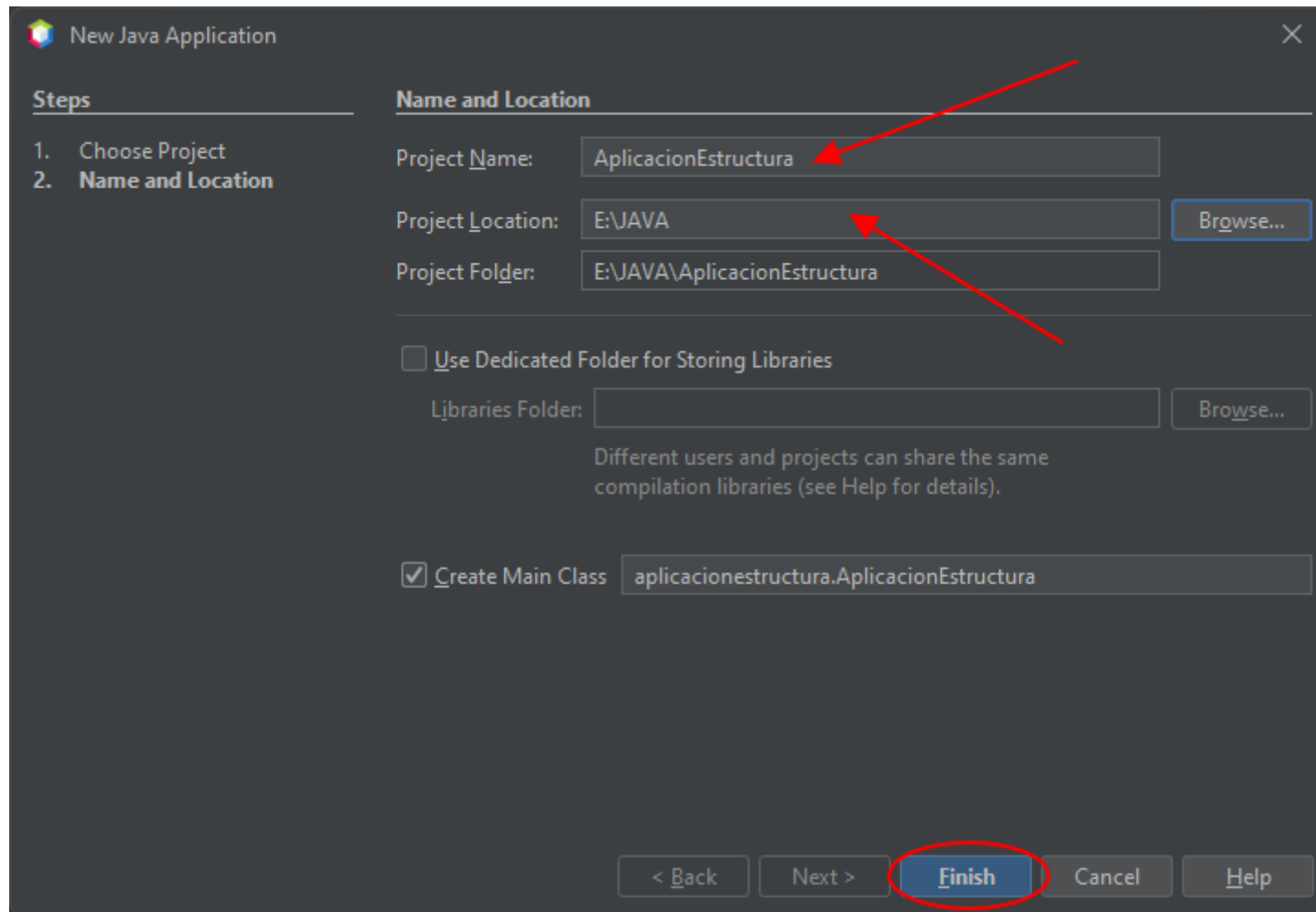
- Estructura y bloques fundamentales.
- Identificadores.
- Palabras reservadas y palabras clave.
- Tipos de datos.
- Variables. Declaración, inicialización y utilización. Almacenamiento en memoria.
- Literales.
- Constantes.
- Operadores y expresiones. Precedencia de operadores.
- Conversiones de tipo. Implícitas y explícitas (casting).
- Comentarios.

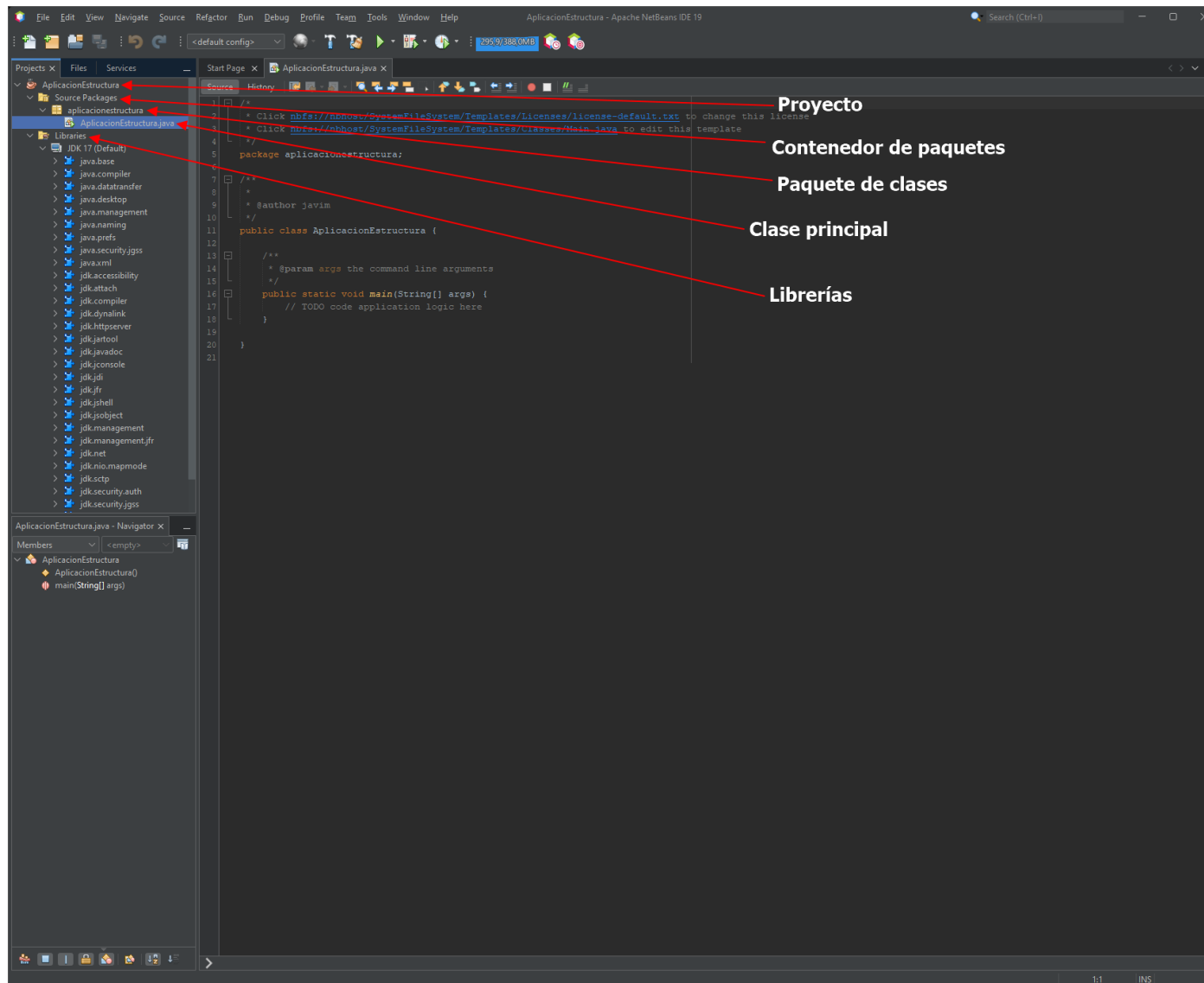
1. ESTRUCTURA Y BLOQUES FUNDAMENTALES

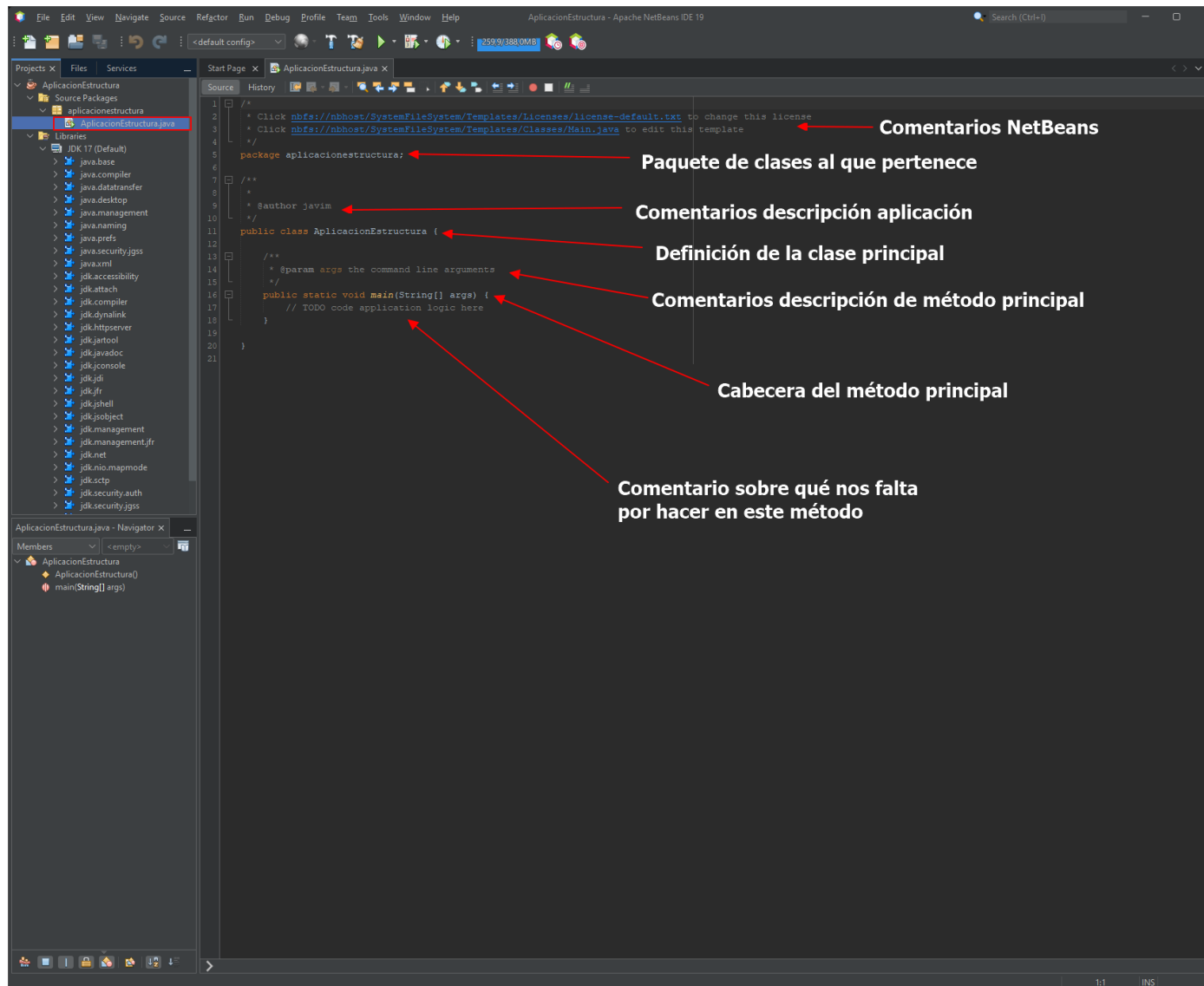












2. IDENTIFICADORES



Podemos definir los identificadores como los nombre que se asigna a los distintos elementos de un programa para identificarlos con respecto al resto.

Los identificadores en Java deben seguir las siguientes normas:

- Debe empezar por una letra, el símbolo \$ o subrayado (_).
- Está constituido por una combinación de letras y/o números.
- Existe distinción entre mayúsculas y minúsculas (Case Sensitive).
- No se recomienda utilizar palabras acentuadas ni el uso del carácter ñ.

Además, se aconseja:

- Cuando un identificador esté formado por dos o más palabras pasaremos a mayúsculas el primer carácter de la segunda palabra en adelante, por ejemplo: **resultadoSuma** (camel case).



- Si el identificador se encarga de dar nombre a una clase o a un interfaz lo denominaremos comenzando con una letra en mayúsculas, por ejemplo: `class AltaUsuario` (Pascal case).
- Al definir constantes usaremos siempre palabras con todas las letras en mayúsculas, por ejemplo: `PI`.

Es de gran importancia escoger identificadores claros y, como su nombre indica, que identifiquen adecuadamente la variable, constante, función, etc. Nunca podemos utilizar como identificadores las palabras reservadas del lenguaje.

3. PALABRAS RESERVADAS Y PALABRAS CLAVE



Las palabras reservadas y las palabras clave son identificadores especiales, que el compilador identifica como parte del lenguaje de programación. Estas palabras reservadas y clave se escriben siempre en minúscula, debido a que Java es un lenguaje case sensitive, y no pueden ser utilizadas como identificadores para variables, constantes, métodos, etc.

Las palabras reservadas y clave del lenguaje Java son las siguientes:

Reserved Words

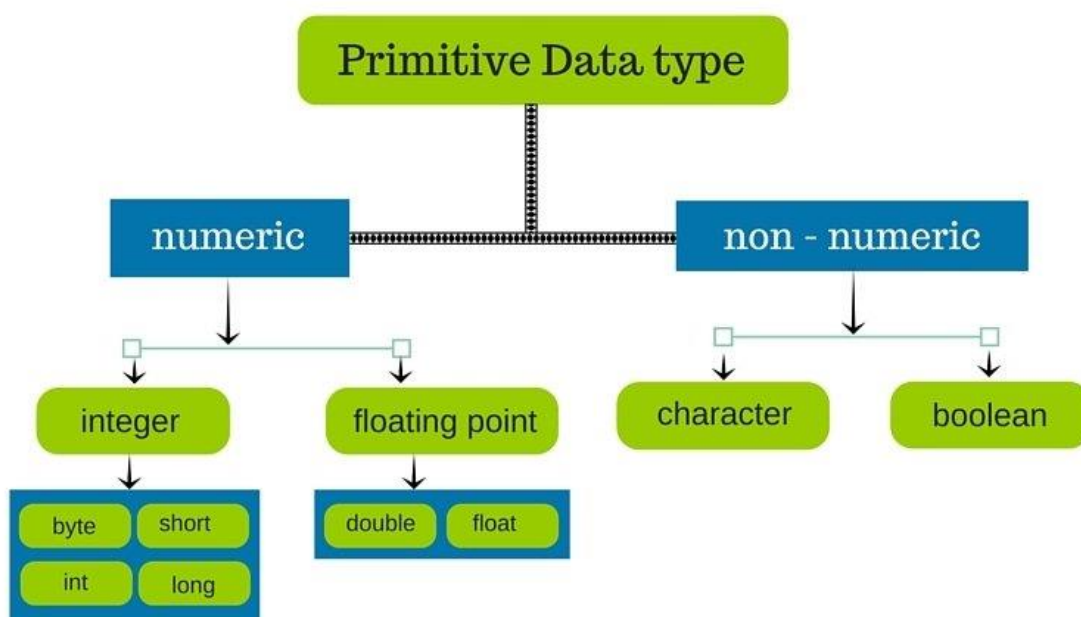
The following words are reserved for the EL and should not be used as identifiers:

<code>and</code>	<code>or</code>	<code>not</code>	<code>eq</code>
<code>ne</code>	<code>lt</code>	<code>gt</code>	<code>le</code>
<code>ge</code>	<code>true</code>	<code>false</code>	<code>null</code>
<code>instanceof</code>	<code>empty</code>	<code>div</code>	<code>mod</code>

List of Java Keywords				
Primitive Types and void	Modifiers	Declarations	Control Flow	Miscellaneous
1.boolean	1.public	1.class	1.if	1.this
2.byte	2.protected	2.interface	2.else	2.new
3.char	3.private	3.enum	3.try	3.super
4.short	4.abstract	4.extends	4.catch	4.import
5.int	5.static	5.implements	5.finally	5.instanceof
6.long	6.final	6.package	6.do	6.null
7.float	7.transient	7.throw	7.while	7.true
8.double	8.volatile		8.for	8.false
9.void	9.synchronized		9.continue	9.strictfp
	10.native		10.break	10.assert
			11.switch	11._ (underscore)
			12.case	12.goto
			13.default	13.const
			14.throw	
			15.return	

4. TIPOS DE DATOS

Los tipos de datos se utilizan generalmente para declarar variables y constantes, y son necesarios para que el intérprete o compilador conozca de antemano el tipo de información que va a contener la variable o la constante. Los tipos de datos primitivos en Java son los siguientes.



Tipo de datos	Información representada	Rango	Descripción
byte	Datos enteros	-128 $\longleftrightarrow$ +127	Se utilizan 8 bits (1 byte) para almacenar el dato.
short	Datos enteros	-32768 $\longleftrightarrow$ +32767	Dato de 16 bits de longitud (independientemente de la plataforma).
int	Datos enteros	-2147483648 $\longleftrightarrow$ +2147483647	Dato de 32 bits de longitud (independientemente de la plataforma).
long	Datos enteros	-9223372036854775808 $\longleftrightarrow$ +9223372036854775807	Dato de 64 bits de longitud (independientemente de la plataforma).
char	Datos enteros y caracteres	0 $\longleftrightarrow$ 65535	Este rango es para representar números en unicode, los ASCII se representan con los valores del 0 al 127. ASCII es un subconjunto del juego de caracteres Unicode.
float	Datos en coma flotante de 32 bits	Precisión aproximada de 7 dígitos	Dato en coma flotante de 32 bits en formato IEEE 754 (1 bit de signo, 8 para el exponente y 24 para la mantisa).
double	Datos en coma flotante de 64 bits	Precisión aproximada de 16 dígitos	Dato en coma flotante de 64 bits en formato IEEE 754 (1 bit de signo, 11 para el exponente y 52 para la mantisa).
boolean	Valores booleanos	true/false	Utilizado para evaluar si el resultado de una expresión booleanas es verdadero (true) o falso(false).

2.1. Tipo de datos cadena de caracteres o String



Las cadenas de caracteres, o Strings, son un tipo de dato no primitivo compuesto por una serie de elementos consecutivos, en el que cada uno de estos elementos es un carácter (char). De esta forma podemos representar textos. Por ejemplo: “Esto es una cadena de caracteres”.

5. VARIABLES. DECLARACIÓN, INICIALIZACIÓN Y UTILIZACIÓN. ALMACENAMIENTO EN MEMORIA.

Según la Real Academia de la Lengua Española una variable es “la magnitud que puede tener un valor cualquiera de los comprendidos en un conjunto”. En otras palabras: una variable es una representación, mediante un identificador, de un valor, que puede cambiar durante la ejecución de un programa.

5.1. Declaración

A la hora de declarar una variable en el lenguaje de programación Java, lo primero que debemos hacer es asignarle un tipo de datos, de tal forma que esa variable únicamente va a poder almacenar valores de este tipo de datos.

La estructura sintáctica que debe tener la declaración de una variable comienza por el tipo de dato que le queremos asignar, seguido del identificador de la variable y finalizamos la línea con un punto y coma (;). Por ejemplo:

```
public static void main(String[] args) {  
    int numeroEntero;  
    float numeroPrecisionSimple;  
    double numeroPrecisionDoble;  
    char caracter;  
    boolean booleano;  
    String cadena;  
}
```

Cuando tenemos que declarar varias variables de un mismo tipo, podemos definir las en una misma línea, aunque no suele utilizarse por claridad del código.

```
int numPar, numImpar;
```

5.1.1. Importancia de la elección de los identificadores

Debemos acostumbrarnos a escoger un identificador adecuado para cada elemento. Este identificador debe decirnos por sí mismo qué información está detrás del nombre, de tal forma que solo con leerlos nos hagamos una idea bastante cercana de qué hace el programa.

A continuación, tenemos un ejemplo práctico para que os hagáis una idea de la importancia de escoger un buen identificador para las variables.

Si tuvieseis mucha sed y disponéis de estas dos botellas, ¿de cuál beberías?



¿Y ahora de cuál beberías?



Además, como recomendación adicional, sería conveniente que te acostumbrases a escribir los identificadores en inglés, puesto que en muchas empresas se sigue esta práctica.

5.2. Inicialización

Para inicializar una variable, es decir, proporcionarle el primer dato que va a almacenar lo podemos hacer de dos formas:

La primera consiste en añadir a la definición un símbolo de igual (=), al que llamamos operador de asignación, detrás del nombre de la variable y el valor que le queremos asignar a continuación.

```
public static void main(String[] args) {  
    int numeroEntero = 5;  
    float numeroPrecisionSimple;  
    double numeroPrecisionDoble = 2.4;  
    char caracter;  
    boolean booleano = true;  
    String cadena;  
  
}
```

La segunda consiste en no modificar la definición de la variable, y en otra línea escribiremos el nombre de la variable, el operador de asignación (=), el valor que queremos almacenar y finalizamos con punto y coma (;). (Todas las instrucciones en Java deben terminar con el carácter ;)

```
public static void main(String[] args) {  
    int numeroEntero = 5;  
    float numeroPrecisionSimple;  
    double numeroPrecisionDoble = 2.4;  
    char caracter;  
    boolean booleano = true;  
    String cadena;  
  
    numeroPrecisionSimple = 1.4f;  
    caracter = 'd';  
    cadena = "Hola";  
}
```

5.3. Utilización

Para utilizar una variable lo único que debemos hacer es escribir su nombre dentro de la expresión o instrucción con la que lo vamos a utilizar.

```
public static void main(String[] args) {  
    int numeroEntero = 5;  
    float numeroPrecisionSimple;  
    double numeroPrecisionDoble = 2.4;  
    char caracter;  
    boolean booleano = true;  
    String cadena;  
  
    numeroPrecisionSimple = 1.4f;  
    caracter = 'd';  
    cadena = "Hola";  
  
    int dedosMano = numeroEntero;  
    String saludo;  
    saludo = cadena;  
}
```

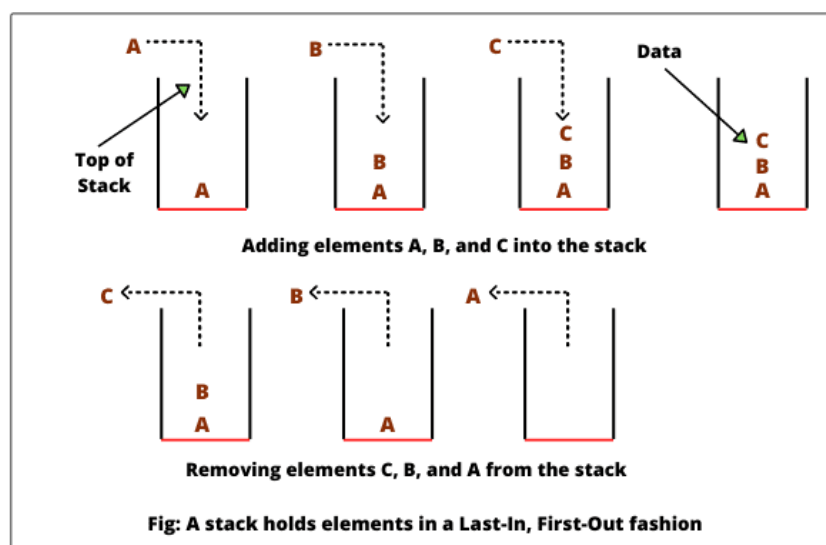
5.4. Almacenamiento



Las variables nos permiten acceder a los datos almacenados en la memoria y existen dos tipos de variables, las primitivas, que son almacenadas en la Memoria Stack y las variables complejas, que son almacenadas en la Memoria Heap.

Puesto que hasta el momento solo estamos trabajando con variables primitivas, nos vamos a centrar en ellas, y cuando lleguemos a las variables complejas, veremos cómo se almacenan en memoria.

Las variables primitivas pueden almacenar datos simples, por ejemplo, números enteros o decimales, caracteres y bytes. Los tipos de datos asociados a las variables primitivas son los tipos de datos primitivos que hemos visto anteriormente. Las variables primitivas retienen valores absolutos mediante la Memoria Stack, la cual se encarga de reservar memoria para cada tipo de dato según los bytes que requiera, por ejemplo, para un entero descrito en 32 bits, debería reservar 4 bytes (1 byte equivale a 8 bits).



A la memoria Stack siempre se le hace referencia en modo LIFO (Last In First Out).

6. LITERALES

En ocasiones utilizaremos directamente los datos en lugar de almacenarlos en memoria. A estos datos se les denominará literales. Estos datos pueden ser:

- De tipo de datos primitivo
- El valor null
- Una cadena de caracteres

Algunos ejemplos son: 'a', 2, 3.45678, true, null, "Me llamo Juan".

7. CONSTANTES

El concepto de constante es similar al de una variable, pero con la particularidad de que no se puede modificar el valor con el que se le inicializa. Por tanto, será una buena práctica utilizarlo, en lugar de una variable, cuando no tengamos previsto cambiar el valor de la misma.

Otra buena práctica es la de nombrar las constantes con identificadores donde todos sus caracteres estén escritos en mayúsculas.

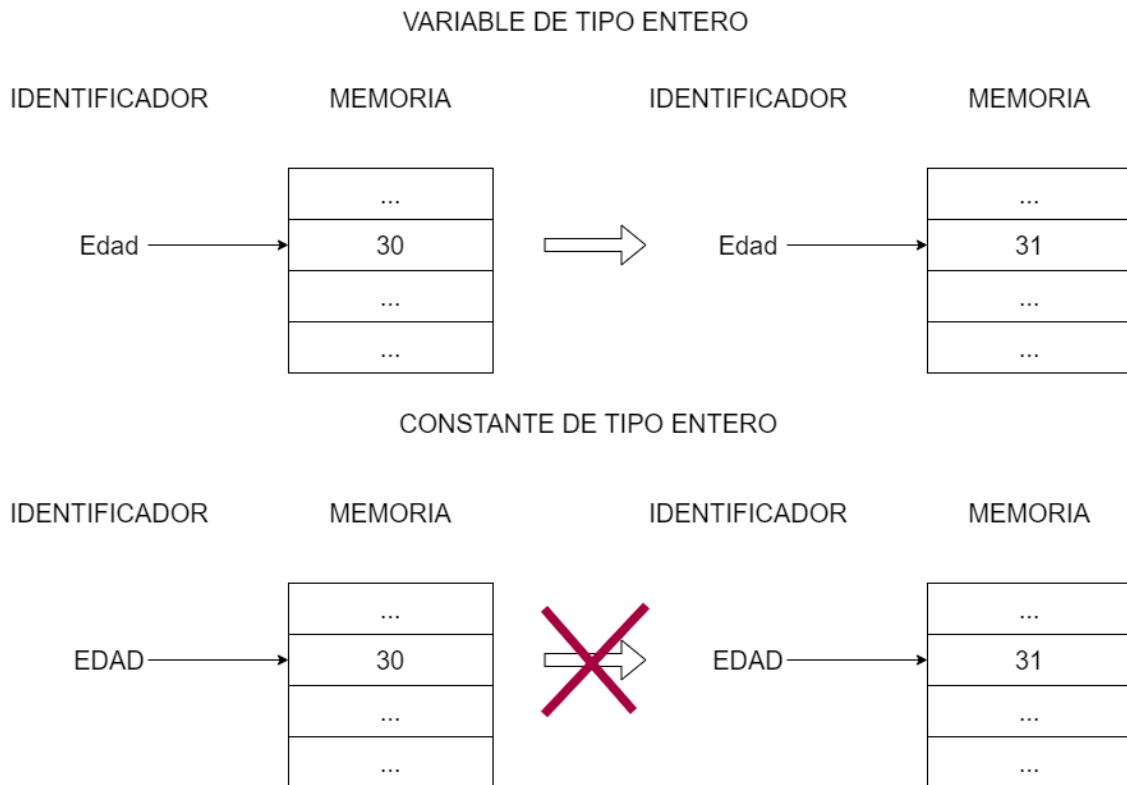
Para crear una constante en nuestro código vamos a necesitar anteponer la palabra clave "final" al tipo de dato de la constante, como se ve en el siguiente ejemplo:

```
final byte DIASSEMANA = 7;
```

Las constantes se inicializan y se utilizan igual que una variable, salvo que únicamente puede asignarse una vez el valor al que hace referencia.

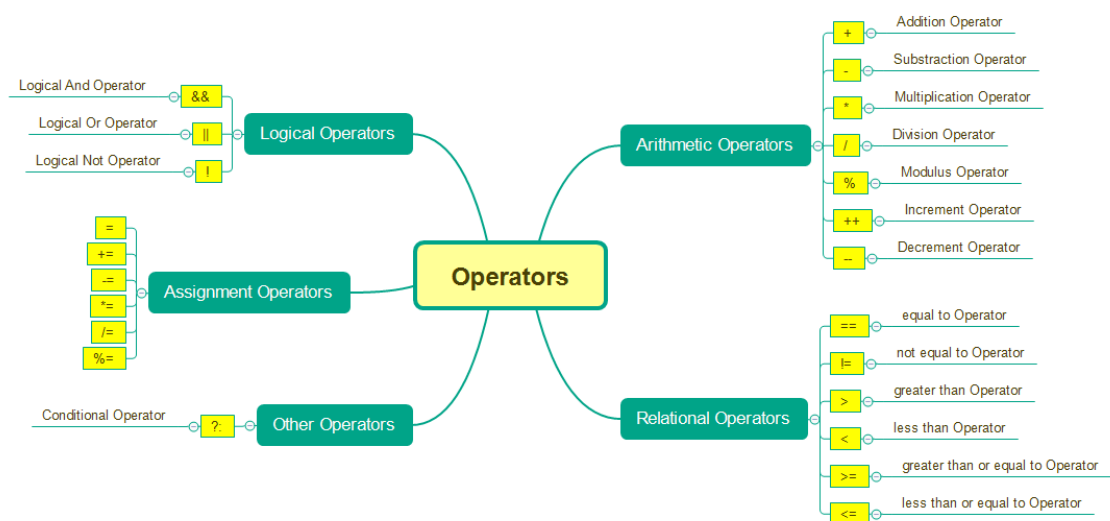
Es decir, no podríamos hacer lo siguiente:

```
final byte DIASSEMANA = 7;  
DIASSEMANA = 8;
```



8. OPERADORES Y EXPRESIONES

Los operadores son acciones que podemos realizar con uno o varios operandos para conseguir un resultado. En el lenguaje Java disponemos de distintos tipos de operadores:



8.1. Operador de asignación

Este operador se simboliza con el símbolo igual (=). Con el asignamos valores a las variables e inicializamos a las constantes.

```
final byte DIASSEMANA = 7;
String mensaje = "Hola, bienvenido";
byte copiaDias;

copiaDias = DIASSEMANA;
```

En el lenguaje Java se nos permite concatenar varios operadores de asignación para asignar el mismo valor a varias variables o constantes a la vez, pero no suele ser lo habitual, por claridad del código.

```
int numeroInicio = numeroActual = 0;
```

8.2. Operadores aritméticos

Los operadores aritméticos con los que trabaja el lenguaje Java son los siguientes:

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Como podemos ver, tenemos dos tipos: unarios, es decir, que solo necesitan un operando y binarios, que necesitan dos operadores.

```
int resultadoSuma = 5 + 7;  
int contador = 0;  
++contador;
```

Los valores unarios vamos a poder utilizarlos tanto delante como detrás de la variable, aunque dependiendo de en la situación en la que lo utilicemos esto puede modificar el resultado. Cuando lo utilizamos únicamente con el operador no afectará al resultado, pero si lo combinamos con una asignación, el resultado que obtendremos será distinto.

En el siguiente ejemplo los dos contadores terminarán con el mismo valor, 1.

```
int contador01 = 0;  
int contador02 = 0;  
++contador01;  
contador02++;
```

Mientras que en el siguiente ejemplo el contador01 terminará con el valor 1, mientras que el contador02 terminará con el valor 0.

```
int contador01 = 0;  
int contador02 = 0;  
contador01 = ++contador01;  
contador02 = contador02++;
```

Esto es debido a que si ponemos los operadores ++ o -- delante de una variable tras un operador de asignación, primero se realiza la operación de aumento o decremento y después se realiza la asignación.

Pero si ponemos los operadores ++ o -- detrás de una variable tras un operador de asignación, primero se realiza la operación de asignación y después se realiza el incremento o decremento.

8.3. Operadores de asignación y operación

Los operadores de asignación y operación combinan ambos tipos de operadores para llevar a cabo ambas acciones utilizando un único operador. Disponemos de los siguientes operadores de este tipo:

Operator	Purpose	Example	Equivalent
+=	Addition	x += 2	x = x + 2
-=	Subtraction	x -= 2	x = x - 2
/=	Division	x /= 2	x = x / 2
*=	Multiplication	x *= 2	x = x * 2
%=	Modulus	x %= 2	x = x % 2

Podemos hacer uso de ellos cuando el primer operando de la operación aritmética coincide con la variable donde vamos a almacenar el resultado.

Podríamos modificar el ejercicio de la práctica 02 sobre el área de un triángulo utilizando el operador de división y asignación, y nos devolvería el mismo resultado.

```
public void main(String[] args) {  
    int base = 25;  
    int altura = 100;  
    int area;  
  
    area = 25 * 100;  
    area /= 2;  
}
```

8.4. Operadores relacionales

Son los operadores a través de los cuales vamos a comparar unos operandos con otros, siempre teniendo en cuenta que son del mismo tipo de dato. El resultado de estos operadores será un valor booleano, true en el caso de que la condición sea cierta y false en el caso de que no lo sea.

Operator	Use	Description
>	op1 > op2	op1 is greater than op2
>=	op1 >= op2	op1 is greater than or equal to op2
<	op1 < op2	op1 is less than op2
<=	op1 <= op2	op1 is less than or equal to op2
==	op1 == op2	op1 and op2 are equal
!=	op1 != op2	op1 and op2 are not equal

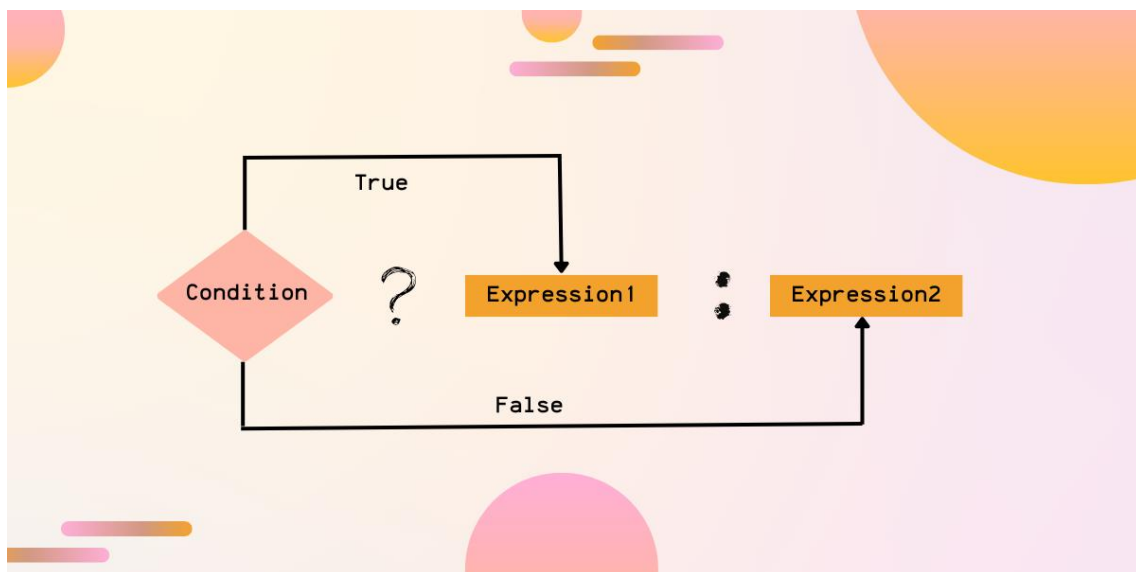
```
public void main(String[] args) {
    int edadAlumno01 = 19;
    int edadAlumno02 = 24;
    boolean esMayor;

    esMayor = edadAlumno01 > edadAlumno02;
}
```

8.5. Operador ternario

El operador ternario es una estructura empleada para realizar operaciones condicionales.

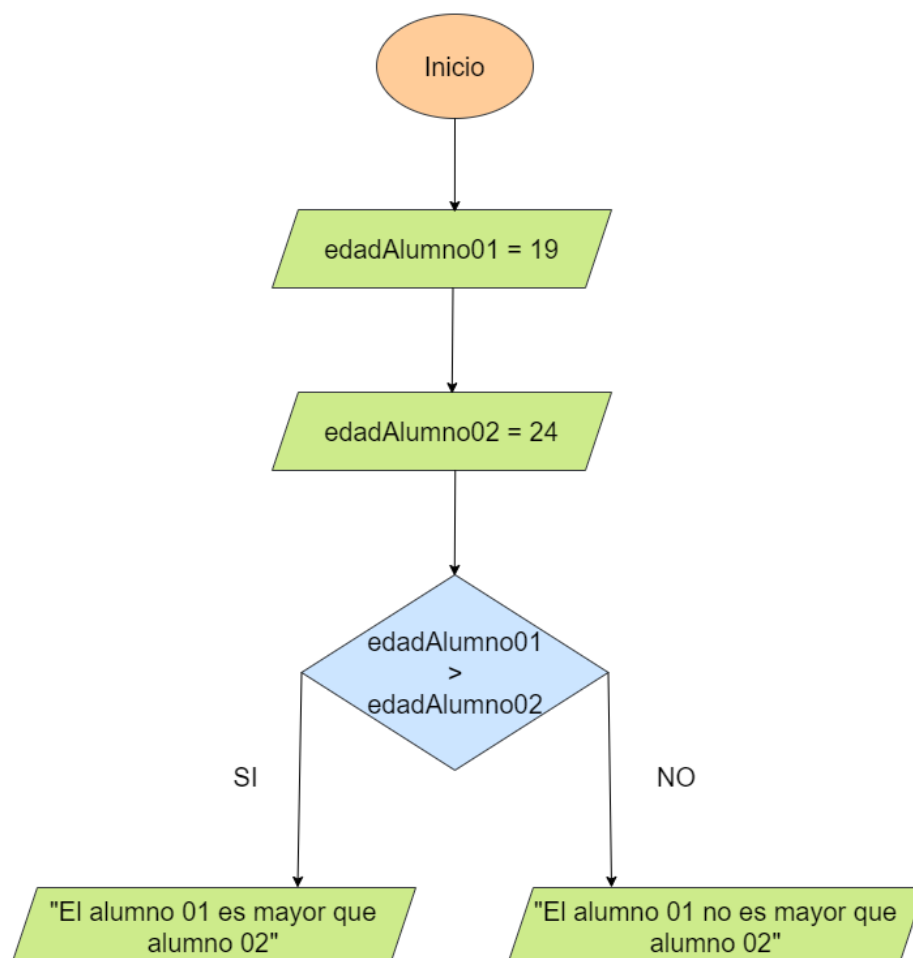
Gracias a él podremos escoger entre dos opciones en función del resultado de la expresión que evaluemos.



Quizás con el vídeo de ejemplo de operador ternario del aula virtual lo entiendas mejor...

Continuando con el ejemplo de la comparación de edades de dos alumnos, ahora vamos a poder crear un resultado con texto para saber si el primer alumno es mayor que el segundo.

```
public void main(String[] args) {  
    int edadAlumno01 = 19;  
    int edadAlumno02 = 24;  
    boolean esMayor;  
    String resultado;  
  
    esMayor = edadAlumno01 > edadAlumno02;  
  
    resultado = (esMayor) ? "El alumno 01 es mayor que el alumno 02" :  
    "El alumno 01 no es mayor que el alumno 02";  
}
```



8.6. Operadores Lógicos

Los operadores lógicos nos permiten realizar operaciones lógicas, es decir, operar con los valores booleanos true y false. A continuación mostramos los distintos operadores.

Logical Operators in Java

- Translating logic into Java

AND &&

OR ||

XOR ^

NOT !

April 6, 1998

CS102-02 Lecture 2-1

Y los resultados de estas operaciones corresponden con los de esta tabla:

A	B	A B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

Aquí tenemos un ejemplo de cómo se utilizan

```
boolean opinionJuan = true;
boolean opinionMaria = false;
boolean resultadoDiscusion;

resultadoDiscusion = opinionMaria && opinionJuan;
```

8.7. Precedencia de operadores

Existen ocasiones en las que combinamos distintos operadores y operandos, a esto se le denomina expresiones.

Existen unas normas de precedencia, es decir, de orden de realización de operaciones cuando combinamos distintos operadores en una expresión. A continuación, tenemos la tabla que nos indica de la parte superior a la inferior, los operadores que tienen más preferencia a los que tienen menos.

Operator	Category	Precedence
Unary Operator	postfix	expression++ expression--
	prefix	++expression --expression +expression -expression ~!
Arithmetic Operator	multiplication	* / %
	addition	+ -
Shift Operator	shift	<< >> >>>
Relational Operator	comparison	< > <= >= instanceof
	equality	== !=
Bitwise Operator	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	
Logical Operator	logical AND	&&
	logical OR	
Ternary Operator	ternary	? :
Assignment Operator	assignment	= += -= *= /= %= &= ^= = <<=
		>>= >>>=

9. CONVERSIONES DE TIPOS. IMPLÍCITAS Y EXPLÍCITAS (CASTING)

Como hemos visto, todas las variables de Java tienen asociado un tipo. Cuando asignamos un valor a una variable, ambos deben ser del mismo tipo.

```
int numEntero = 2;
```

```
double numComaFlotante = 2.3;
```

Cada tipo se caracteriza por ocupar un tamaño en memoria, donde se almacenan los valores correspondientes.

Si intentamos asignar un valor con un tamaño mayor a otro de un tamaño menor el compilador nos avisará de que estamos cometiendo un error y no nos dejará ejecutar. Por ejemplo:

```
double numComaFlotante = 2.3;
```

```
int numEntero = numComaFlotante;
```

La razón de este control de tipos tan estricto es evitar errores durante la ejecución del programa, ya que es evidente que una variable de un tipo no puede almacenar valores con un tamaño superior.

En el ejemplo anterior un valor double ocupa 64 bit, mientras que un int usa 32 bits para almacenar un valor. Por tanto, el valor double no cabe en el espacio de memoria reservado para un int.

Sin embargo, Java sí que nos permite guardar un valor int en una variable double sin problema. Por ejemplo:

```
int numEntero = 2;
```

```
double numComaFlotante = numEntero;
```

Para poder hacer esto, Java convierte de forma automática el valor entero 2 en el valor double 2.0 antes de asignarlo a la variable numComaFlotante. Esto es posible porque el tamaño de double es mayor que el de int. Es decir, este tipo de

conversiones y asignaciones automáticas será posible cuando la variable sea de un tamaño mayor que el valor asignado. A este tipo de operación lo conocemos como conversiones implícitas o por ensanchamiento.

byte => short => int => long => float => double

En el caso de interesarnos almacenar un valor con un tamaño mayor al de una variable, Java dispone de una herramienta para llevarlo a cabo, pero con una pérdida de información. Esta herramienta es a lo que denominamos casting, y nos permitirá realizar conversiones explícitas o por estrechamiento. Tomando el ejemplo que vimos anteriormente, le incluimos el casting para que sea posible almacenar un valor double en un int, pero perdiendo los decimales:

```
double numComaFlotante = 2.3;
```

```
int numEntero = (int) numComaFlotante;
```

El valor actual de numEntero es 2.

El casting consiste en poner delante del valor el tipo al que queremos convertirlo entre paréntesis.

La relación de conversiones sería la inversa a la que veíamos en las conversiones por ensanchamiento:

double => float => long => int => short => byte

Existe un caso especial de conversión, los tipo char, que, con su tamaño de 16 bits, pueden convertirse a su valor entero en el código UNICODE. Esto permite asignarlos a una variable int:

```
int caracter = 'a';
```

En este caso la variable caracter almacenaría el valor 97.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL	(null)	32	20	040	 Space	64	40	100	@ @		96	60	140	` <b>`	
1	1	001	SOH	(start of heading)	33	21	041	! !	65	41	101	A A		97	61	141	a a	
2	2	002	STX	(start of text)	34	22	042	" "	66	42	102	B B		98	62	142	b b	
3	3	003	ETX	(end of text)	35	23	043	# #	67	43	103	C C		99	63	143	c c	
4	4	004	EOT	(end of transmission)	36	24	044	$ \$	68	44	104	D D		100	64	144	d d	
5	5	005	ENQ	(enquiry)	37	25	045	% %	69	45	105	E E		101	65	145	e e	
6	6	006	ACK	(acknowledge)	38	26	046	& &	70	46	106	F F		102	66	146	f f	
7	7	007	BEL	(bell)	39	27	047	' '	71	47	107	G G		103	67	147	g g	
8	8	010	BS	(backspace)	40	28	050	((	72	48	110	H H		104	68	150	h h	
9	9	011	TAB	(horizontal tab)	41	29	051	))	73	49	111	I I		105	69	151	i i	
10	A	012	LF	(NL line feed, new line)	42	2A	052	* *	74	4A	112	J J		106	6A	152	j j	
11	B	013	VT	(vertical tab)	43	2B	053	+ +	75	4B	113	K K		107	6B	153	k k	
12	C	014	FF	(NP form feed, new page)	44	2C	054	, ,	76	4C	114	L L		108	6C	154	l l	
13	D	015	CR	(carriage return)	45	2D	055	- -	77	4D	115	M M		109	6D	155	m m	
14	E	016	SO	(shift out)	46	2E	056	. .	78	4E	116	N N		110	6E	156	n n	
15	F	017	SI	(shift in)	47	2F	057	/ /	79	4F	117	O O		111	6F	157	o o	
16	10	020	DLE	(data link escape)	48	30	060	0 0	80	50	120	P P		112	70	160	p p	
17	11	021	DC1	(device control 1)	49	31	061	1 1	81	51	121	Q Q		113	71	161	q q	
18	12	022	DC2	(device control 2)	50	32	062	2 2	82	52	122	R R		114	72	162	r r	
19	13	023	DC3	(device control 3)	51	33	063	3 3	83	53	123	S S		115	73	163	s s	
20	14	024	DC4	(device control 4)	52	34	064	4 4	84	54	124	T T		116	74	164	t t	
21	15	025	NAK	(negative acknowledge)	53	35	065	5 5	85	55	125	U U		117	75	165	u u	
22	16	026	SYN	(synchronous idle)	54	36	066	6 6	86	56	126	V V		118	76	166	v v	
23	17	027	ETB	(end of trans. block)	55	37	067	7 7	87	57	127	W W		119	77	167	w w	
24	18	030	CAN	(cancel)	56	38	070	8 8	88	58	130	X X		120	78	170	x x	
25	19	031	EM	(end of medium)	57	39	071	9 9	89	59	131	Y Y		121	79	171	y y	
26	1A	032	SUB	(substitute)	58	3A	072	: :	90	5A	132	Z Z		122	7A	172	z z	
27	1B	033	ESC	(escape)	59	3B	073	; ;	91	5B	133	[[		123	7B	173	{ {	
28	1C	034	FS	(file separator)	60	3C	074	< <	92	5C	134	\ \		124	7C	174	| 	
29	1D	035	GS	(group separator)	61	3D	075	= =	93	5D	135	]]		125	7D	175	} }	
30	1E	036	RS	(record separator)	62	3E	076	> >	94	5E	136	^ ^		126	7E	176	~ ~	
31	1F	037	US	(unit separator)	63	3F	077	? ?	95	5F	137	_ _		127	7F	177	 DEL	

Source: www.LookupTables.com

10. COMENTARIOS

Los comentarios son código que el compilador ignora, pero que nos son de utilidad a la hora de documentar nuestras aplicaciones.

En el lenguaje Java disponemos de dos formas de introducir comentarios en el código:

1. **Línea simple:** colocamos dos barras inclinadas “//” y todo lo que se encuentre detrás de ellas, hasta el final de la línea, se considerará un comentario. Ejemplo:

```
// Esto es un comentario de una sola línea
```

2. **Multilínea:** el comentario comienza con los símbolos barra inclinada y asterisco “/*” y finaliza con asterisco y barra inclinada “*/”, sin importar que se encuentren en distintas líneas. Ejemplo:

```
/*Así se utilizan
```

```
los comentarios multilínea*/
```

El uso de los comentarios ha ido variando a lo largo de los años... en un primer momento no se utilizaban,... después se consideraron de gran importancia y se llegó a tener códigos en los que una línea era una instrucción y la siguiente un comentario... y actualmente se recomienda hacer un uso controlado de los mismos apoyándonos principalmente en identificadores autodescriptivos, de tal forma que solo necesitemos introducir comentarios cuando la lógica de una determinada parte del código sea poco entendible.